Reti di Calcolatori

Michela Maria Tasca
Prof. Rundo — UniCt

A.A. 2025/2026

Indice

1 Introduzione alle Reti di Calcolatori **2**
1.1 Struttura e Componenti di una Rete . . . 2
1.1.1 Commutazione di Pacchetto e Instradamento . . . 2
1.1.2 Protocolli di Comunicazione . . . 3
1.1.3 Architettura a Livelli ed Incapsulamento . . . 3
1.1.4 La Pila dei Protocolli (Stack) . . . 3
1.1.5 Il Concetto di Incapsulamento . . . 3
1.2 Modelli di Riferimento: ISO/OSI e TCP/IP . . . 4
1.2.1 Il Modello ISO/OSI . . . 4
1.2.2 Il Modello TCP/IP . . . 4
1.3 Comunicazione e Segnali . . . 4
1.3.1 Trasmissione Fisica . . . 4
1.3.2 Tipologie di Connessione . . . 4
1.4 Topologie e Classificazione delle Reti . . . 5
1.4.1 Tipologie di Trasmissione . . . 5
1.4.2 Topologie Fisiche . . . 5
1.4.3 Classificazione per Estensione Geografica . . . 5
1.5 Sicurezza nelle Reti . . . 5

2 Application Layer **7**
2.1 Paradigmi di Architettura di Rete . . . 7
2.1.1 Architettura Client-Server . . . 7
2.1.2 Architettura Peer-to-Peer (P2P) . . . 8
2.2 Comunicazione tra Processi . . . 8
2.3 Socket . . . 9
2.4 Requisiti delle Applicazioni di Rete . . . 9
2.5 Protocolli del Livello di Trasporto . . . 9
2.5.1 TCP (Transmission Control Protocol) . . . 9
2.5.2 UDP (User Datagram Protocol) . . . 10
2.6 Applicazioni Note . . . 10
2.6.1 Telnet e SSH . . . 10
2.6.2 FTP (File Transfer Protocol) . . . 10
2.6.3 HTTP (HyperText Transfer Protocol) . . . 11

Capitolo 1

Introduzione alle Reti di Calcolatori

1.1 Struttura e Componenti di una Rete

Definizione 1.1: Rete Informatica

Una rete informatica è un sistema che permette il collegamento e la comunicazione tra più dispositivi indipendenti.

Un sistema di comunicazione si compone fondamentalmente di due dimensioni interconnesse:

- **Parte Fisica**: Costituita dall'infrastruttura hardware che connette i dispositivi. I nodi sono collegati tramite **communication link** (collegamenti di comunicazione) e **packet switch** (commutatori di pacchetto).
  - **Mezzi fisici**: I collegamenti possono utilizzare cavi di rame, fibre ottiche o onde elettromagnetiche.
  - **Velocità di trasmissione**: Viene misurata in **bps** (bit per secondo).
- **Parte Logica**: Rappresenta l'insieme di regole e software che permettono la trasmissione e la ricezione dei dati attraverso il mezzo fisico.

1.1.1 Commutazione di Pacchetto e Instradamento

Nelle reti moderne, le informazioni non vengono inviate come un unico flusso continuo, ma vengono divise in unità più piccole chiamate **pacchetti**.

- **Composizione del Pacchetto**: Ad ogni frammento di dati viene aggiunta un'**intestazione** (header) contenente informazioni di controllo.
- **Packet Switch**: Dispositivi come **router** o **link layer switch** ricevono i pacchetti su un collegamento in ingresso e li inoltrano su uno in uscita.
- **Route (Percorso)**: La sequenza di collegamenti e commutatori attraversati da un pacchetto dalla sorgente alla destinazione.
- **Accesso alla Rete**: Gli utenti accedono a Internet tramite gli **ISP** (Internet Service Provider).

1.1.2 Protocolli di Comunicazione

Definizione 1.2: Protocollo

Un protocollo definisce il formato e l'ordine dei messaggi scambiati tra due o più entità in comunicazione, specificando le azioni da intraprendere durante la trasmissione, la ricezione o in risposta a determinati eventi.

I due protocolli fondamentali dell'architettura Internet sono il **TCP** (Transmission Control Protocol) e l'**IP** (Internet Protocol). Lo sviluppo degli standard è affidato alla **IETF** (Internet Engineering Task Force), che pubblica i dettagli tecnici tramite i documenti **RFC** (Request For Comment).

1.1.3 Architettura a Livelli ed Incapsulamento

Per gestire la complessità della progettazione hardware e software, le reti sono strutturate in **livelli** (layer). Questo approccio modulare facilita l'aggiornamento dei componenti senza dover modificare l'intero sistema.

1.1.4 La Pila dei Protocolli (Stack)

La gerarchia standard si compone di 5 livelli principali:

1. **Livello di Applicazione**: Sede delle applicazioni di rete (es. browser, email).
2. **Livello di Trasporto**: Gestisce il trasferimento dei messaggi applicativi tra gli endpoint.
3. **Livello di Rete**: Responsabile del trasferimento dei pacchetti (datagrammi) da un host all'altro attraverso la rete.
4. **Livello di Collegamento (Data Link)**: Gestisce l'instradamento dei frame tra nodi adiacenti della rete.
5. **Livello Fisico**: Si occupa della trasmissione dei singoli bit sul mezzo fisico.

1.1.5 Il Concetto di Incapsulamento

L'incapsulamento è il processo mediante il quale ogni livello aggiunge informazioni di controllo ai dati provenienti dal livello superiore.

- **Payload**: Il carico di dati utile ricevuto dal livello precedente.
- **Header**: Un pre-dato di formattazione che prepara il ricevitore alla gestione dell'informazione successiva.

Processo di trasmissione (Origine):

- **Messaggio (M)**: Generato al livello Applicazione.
- **Segmento**: Il livello Trasporto aggiunge l'header H_t (necessario per la consegna alla corretta istanza applicativa).
- **Datagramma**: Il livello Rete aggiunge l'header H_n (per l'instradamento tra host).

- **Frame**: Il livello Collegamento aggiunge l'header H_l (per la consegna fisica tra nodi).

1.2 Modelli di Riferimento: ISO/OSI e TCP/IP

1.2.1 Il Modello ISO/OSI

È un modello teorico di riferimento costituito da 7 livelli:

- **Host Layers** (Gestione end-to-end): Application, Presentation (rappresentazione e cifratura dati), Session (comunicazione inter-host), Transport.
- **Media Layers** (Gestione del transito): Network, Data Link, Physical.

1.2.2 Il Modello TCP/IP

Utilizzato nelle implementazioni reali, semplifica la struttura in 4 livelli:

- **Application Layer**: Accorpa i livelli Application, Presentation e Session dell'OSI.
- **Transport Layer**: Gestisce la connessione tra processi.
- **Network Layer**: Gestisce l'indirizzamento logico (IP).
- **Network Access Layer**: Accorpa i livelli Data Link e Physical (gestione del mezzo fisico e indirizzamento MAC).

1.3 Comunicazione e Segnali

1.3.1 Trasmissione Fisica

I dati sono processati dalle CPU in linguaggio binario. La distinzione tra 0 e 1 avviene tramite soglie di tensione:

- **Bit 1**: Tensione $> 3.3V$.
- **Bit 0**: Tensione $< 0.8V$.
- **Area Intermedia**: Zona di incertezza da evitare per prevenire errori di interpretazione.

Problematiche del segnale:

- **Distorsione**: Causata dalla distanza di propagazione e dai ritardi di induzione del mezzo fisico.
- **Ritardo di commutazione**: Il segnale non può cambiare stato istantaneamente da 0 a 1.

1.3.2 Tipologie di Connessione

- **Connectionless**: Invio dei dati senza previa instaurazione di una connessione e senza conferma di ricezione (es. invio di una lettera ordinaria).

- **Connection-oriented**: Prevede segnali di **ACK** (Acknowledge) per confermare la corretta ricezione.
- **Affidabilità**: Necessaria in ambiti critici (medico, militare). Si ottiene spesso tramite il **Timing**: se un ACK non arriva entro un tempo limite, il dato viene reinviato.

1.4 Topologie e Classificazione delle Reti

1.4.1 Tipologie di Trasmissione

- **Point-to-Point**: Comunicazione diretta tra due dispositivi.
- **Broadcast**: Un dispositivo invia segnali a tutti i dispositivi della rete. Richiede la gestione delle **collisioni**.

1.4.2 Topologie Fisiche

- **Bus**: Semplice per il broadcast, ma richiede **arbitri di bus** per gestire le collisioni se più sistemi trasmettono contemporaneamente.
- **Anello**: Richiede la definizione di un orientamento per la circolazione del segnale.
- **Stella**: I nodi sono collegati a un centro comune; la gestione di più pacchetti avviene tramite temporizzazione.

1.4.3 Classificazione per Estensione Geografica

- **PAN** (Personal Area Network): Raggio $\sim$ 1 metro.
- **LAN** (Local Area Network): Raggio fino a 1 km.
- **MAN** (Metropolitan Area Network): Raggio fino a 10 km.
- **WAN** (Wide Area Network): Raggio da 100 a 1000 km.
- **Internet**: WAN planetaria con raggio di circa 10000 km.

1.5 Sicurezza nelle Reti

Le architetture distribuite sono soggette a diverse minacce:

- **Malware**:
  - **Virus**: Oggetti dannosi legati a file eseguibili.
  - **Worm**: Software in grado di autoreplicarsi tra computer.
  - **Botnet**: Reti di calcolatori compromessi controllate da remoto.
- **Attacchi di Rete**:
  - **DoS (Denial of Service)**: Esaurimento delle risorse per rendere un servizio indisponibile.

- **Packet Sniffing**: Intercettazione dei pacchetti in transito.
- **IP Spoofing**: Falsificazione dell'indirizzo IP mittente.

Capitolo 2

Application Layer

Il livello di applicazione modella le applicazioni di rete e le modalità di comunicazione tra le varie strutture che compongono il sistema. In questo contesto, la comunicazione avviene tra processi eseguiti su calcolatori diversi (nodi della rete).
È il livello più alto nello stack dei protocolli, ed è quello più vicino all'utente. È infatti il livello delle **applicazioni di rete**: la vastità delle funzionalità offerte dalle reti, sono alla base del successo di Internet.

2.1 Paradigmi di Architettura di Rete

Le applicazioni di rete si basano principalmente su due paradigmi organizzativi per gestire l'interazione tra gli host:

2.1.1 Architettura Client-Server

Nell'architettura **Client-Server**, i ruoli dei nodi sono ben definiti e asimmetrici:

- **Server**:
  - È un host **sempre attivo** e connesso alla rete che risponde alle richieste del client
  - Possiede un **indirizzo IP statico** (permanente) per facilitare la reperibilità da parte dei client.
  - Può essere costituito da un singolo calcolatore o da un intero **data center** scalabile.
  - **Sicurezza**: L'IP statico è vulnerabile ad attacchi diretti una volta scoperto.
- **Client**:
  - Manda richieste al server
  - Si connette al server solo quando necessita di un servizio.
  - Può avere un **indirizzo IP dinamico**.
  - I client non comunicano mai direttamente tra loro in questo modello.

Esempi di protocolli che utilizzano questo paradigma sono **HTTP** (web), **IMAP** (email) e **FTP** (trasferimento file). In particolare, IMAP permette la visualizzazione delle email memorizzate sul server senza scaricarle localmente, garantendo la sincronizzazione ma rendendo l'utente dipendente dalla disponibilità del server.

2.1.2 Architettura Peer-to-Peer (P2P)

Nell'architettura **Peer-to-Peer**, il sistema sfrutta la comunicazione diretta tra coppie di host chiamati **peer**.

- ogni client (**peer**) ha lo stesso ruolo e stesse capacità
- **Assenza di un arbitro**: Non esiste un server centrale che coordina le comunicazioni.
- **Scalabilità intrinseca**: Ogni nuovo peer che entra nella rete apporta nuove risorse, aumentando la capacità di servizio del sistema senza costi eccessivi di infrastruttura.
- **Svantaggi**: Può causare congestione della rete globale (occupazione di banda) e presenta maggiori rischi di sicurezza, poiché i peer possono essere insicuri o malevoli.

Esempi tipici sono il file sharing P2P e i protocolli come **BitTorrent**.

2.2 Comunicazione tra Processi

Sebbene si parli spesso di comunicazione tra host, a livello logico sono i **processi** in esecuzione sugli host a scambiarsi messaggi.
La comunicazione può avvenire:

- **intra-host:** quando due processi risiedono sullo stesso host utilizzano meccanismi di **Inter-Process Communication IPC** forniti dal sistema operativo. Questi mecccanismi sono altamente efficenti perchè non richiedono l'uso della rete.
- **inter-host:** quando i processi si trovano su host diversi, la comunicazione avviene tramite **scambio di messaggi** attraverso la rete. Questi messaggi sono solitamente incapsulati in protocolli di livello applicativo (HTTP, SMTP, FTP, ecc) e vengono trasportati tramite protocolli di livello inferiore (TCP; UDP). Lo scambio di messaggi serve a:
  - trasferire dati, banalmente
  - sincronizzare i processi, ovvero a coordinare le azioni nel tempo

Perché la comunicazione avvenga correttamente, ogni processo deve essere identificabile univocamente tramite due parametri:

1. **Indirizzo IP**: Identifica l'host (calcolatore) sulla rete (32 bit per IPv4, 128 bit per IPv6).
2. **Numero di Porta**: Identifica lo specifico servizio o processo all'interno dell'host.

Le porte sono suddivise in tre gruppi.

- **Porte Well-known (0-1023)**: Destinate ai servizi di sistema (es. HTTP: 80, HTTPS: 443, DNS: 53).

- **Porte Registrate (1024-49151)**: Assegnate dall'ICANN per usi specifici.
- **Porte Private/Dinamiche (49152-65535)**: Utilizzate per connessioni temporanee.

2.3 Socket

In un calcolatore, il sistema operativo tiene traccia dei processi attraverso il loro PID (*Process ID*). Quando il processo ha necessità di comunicare con un altro processo su un'altra macchina, il PID non è sufficente, visto la sua esistenza solo in locale. Per questo nascono i socket.

Definizione 2.1: Socket

Il **socket** è un'interfaccia software (API) fornita dal sistema operativo che permette a un processo applicativo di inviare e ricevere messaggi attraverso la rete. Funge da interfaccia tra il livello di applicazione e il livello di trasporto.

Di base, "collega" due host comunicanti ed è composto dall'**inidirizzo IP dell'host** e dal **numero di porta** associato al processo.
Nella pratica, il processo chiede al SO di creare il socket, in modo da far identificare la comunicazione dalla coppia IP-porta.

2.4 Requisiti delle Applicazioni di Rete

Le applicazioni hanno esigenze diverse in termini di qualità del servizio.

- **Integrità dei dati**: Alcune applicazioni (es. trasferimento file, email) non tollerano la perdita di dati.
- **Throughput**: Quantità di dati scambiati nell'unità di tempo. Le applicazioni **bandwidth-sensible** (es. video streaming) richiedono una banda minima garantita. Le applicazioni **elastiche** (es. email) si adattano alla banda disponibile.
- **Temporizzazione**: Le applicazioni **time-sensitive** (es. telefonia IP, giochi online) richiedono che i pacchetti arrivino entro un ritardo massimo prefissato.
- **Sicurezza**: include servizi specifici (come la crittocrafia) per proteggere le informazioni

2.5 Protocolli del Livello di Trasporto

Il livello di applicazione si interfaccia con due protocolli principali del livello di trasporto:

2.5.1 TCP (Transmission Control Protocol)

Progettato per garantire una comunicazione affidabile, ordinata e controllata. Ha le seguenti caratteristiche:

- **Affidabile (Reliable)**: Garantisce che i dati arrivino integri e in ordine.

- **Three-Way Handshake**: Stabilisce la connessione tramite tre fasi (SYN, SYN-ACK, ACK), si parla quindi di *connessione oreientata*.
- **Controllo di flusso e congestione**: Regola la velocità di invio per non sovraccaricare il ricevente o la rete.
- **Limitazioni**: Non offre garanzie intrinseche su timing o sicurezza (quest'ultima deve essere gestita dall'applicazione).

2.5.2 UDP (User Datagram Protocol)

Progettato per essere semplice, veloce e minimale. Ha le seguenti caratteristiche:

- **Non affidabile (Unreliable)**: Non garantisce la consegna dei pacchetti.
- **Senza connessione**: Non effettua l'handshake; i dati vengono inviati direttamente.
- **Assenza di controlli**: Non implementa controlli di flusso o congestione.
- **Casi d'uso**: Utilizzato per **VoIP** (Voice Over IP) e streaming, dove la velocità è più importante dell'integrità assoluta del dato.

2.6 Applicazioni Note

2.6.1 Telnet e SSH

Telnet è un protocollo applicativo che permette di connettersi a un computer remoto e operare sulla sua shell locale come se fosse un mirroring del server.

- **Funzionamento**: Utilizza TCP con la porta 23 come destinataria
- . Il client invia input (es. un carattere), il server lo elabora e lo rimanda indietro per visualizzarlo sul terminale locale.
- **Problema Sicurezza**: Telnet invia i dati in **chiaro**. È vulnerabile a *sniffing* e *man-in-the-middle*. Se intercettato, credenziali e dati sensibili sono visibili a terzi.
- **SSH (Secure Shell)**: usa la porta 22. Versione sicura che mitiga i rischi di intercettazione tramite la **three-way handshake** e la **codifica/crittografia** dello stream di dati.

Remote Desktop Protocol è un protocollo con presupposti simili a quelli di Telnet, ma permette la trasmissione dell'intera interfaccia del dispositivo remoto.

2.6.2 FTP (File Transfer Protocol)

Protocollo dedicato al trasferimento di file tra host. Offre un meccanismo di **autenticazione** che avviene dopo lo stabilimento della connessione ma prima di ogni tipo di trasmissione dei dati. A differenza di altri protocolli, FTP utilizza **due connessioni TCP separate**:

- **Canale Comandi (Porta 21)**: Gestito dal **Protocol Interpreter (PI)**. Usato per scambiare comandi (es. PORT, LIST) e risposte.

- **Canale Dati (Porta 20)**: Gestito dal **Data Transfer Process (DTP)**. Usato per il trasferimento effettivo dei file.

Modello di funzionamento dell'FTP: Nel modello operativo, il client apre una porta casuale per i dati e comunica il numero al server tramite il comando **PORT** sul canale comandi. Il server attiva quindi la connessione dati partendo dalla sua porta 20 verso la porta indicata dal client.
Questo protocollo ha però delle vulnerabilità:

- *abilitare l'accesso all'intero file system* tramite FTP è una mossa rischiosa, in quanto bisogna regolare quali porzioni rendere accessibili
- le credenziali di accesso e i dati, in modalità default, sono *trasmessi in chiaro*, rendendo FTP vulnerabile ad attacchi.

SFTP: risolve il problema implementando l'encryption di SSH, usando la porta 22.

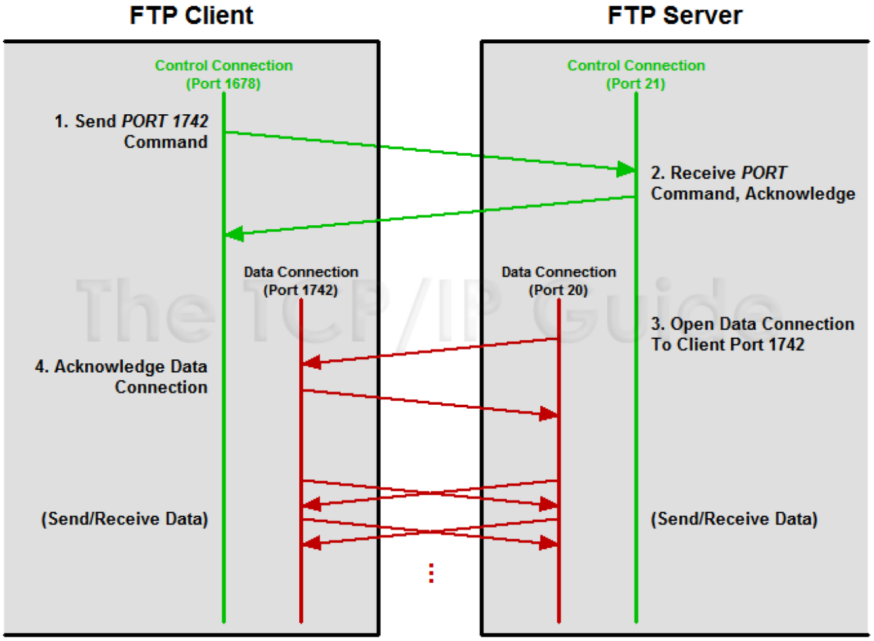

Figura 2.1: Diagramma di interazione

Il diagramma di sequenza mostra l'invio del comando PORT da parte del client sulla connessione di controllo (es. porta 1678 verso porta 21 del server). Successivamente, il server apre la connessione dati (Porta 20 verso Porta 1742 del client) per iniziare il trasferimento effettivo.

2.6.3 HTTP (HyperText Transfer Protocol)

HTTP permette ad un host di **richiedere delle risorse**, chiamate oggetti, ad un Web Server. Questi oggetti possono essere pagine HTML o file multimediali di vario tipo. Per accedere a queste risorse viene utilizzato un URL (Uniform Resource Locator). Esso è quindi il protocollo alla base delle pagine Web, e segue l'architettura ClientServer. Le porte dedicate al protocollo sono:

- **Porta 80** - lato server: è laporta da cui il server riceverà le richieste
- **Porta sorgente** - lato client: stabilita casualmente dal SO.

URL

È composto da varie parti:

- **Protocollo:** indica come deve avvenire la comunicazione (es: https://)
- **Nome di dominio:** Indica dove si trova la risorsa. È associato a un indirizzo IP tramite DNS. (es: www.esempio.com)
- **Porta:** Indica su quale porta del server il browser deve collegarsi.
- **Path:** Indica quale risorsa si vuole sul server. (es: /cartella/pagina.it)
- **Query stringhe:** Sono parametri aggiuntivi inviati al server usati per ricerche, per tracciare e per i login
- **Frammento:** indica una parte specifica della pagina. Non viene inviato al server: serve solo al browser.